Ford Connected Vehicle & Dealer Service Platform

Python + Django + Django REST Framework + PostgreSQL + Docker/Kubernetes — Interview Project & Coding Guide

Important: This is an interview-ready, realistic reference architecture. It is not presented as Ford confidential implementation detail. Describe it as a representative enterprise project inspired by connected-vehicle and dealer-service workflows.

1. The 60-second project story

I worked on a connected-vehicle and dealer-service platform that exposed secure REST APIs for vehicle telemetry, vehicle health, service appointments, diagnostic information and dealer workflows. The backend was built with Python and Django REST Framework. PostgreSQL stored transactional vehicle, customer, dealer and service data. Services were containerized with Docker and deployed to Kubernetes for horizontal scaling and self-healing. API security used OAuth2/JWT-style bearer authentication, role-based authorization, validation, throttling, TLS, secure headers, audit logging and database controls.

2. Business problem

A connected vehicle generates events such as ignition state, odometer changes, diagnostic trouble codes, battery/health signals and service-related alerts. A dealer needs a controlled API layer to retrieve the right vehicle information, create or update service appointments and see actionable diagnostic information. The platform therefore separates external API consumers from internal services and protects customer/vehicle data.

3. Architecture

Layer	Representative components	Purpose
Client	Mobile app / web portal / dealer application	Calls APIs
Edge	DNS → Load Balancer / API Gateway → WAF	TLS termination, routing, protection
API	Django + DRF pods	Authentication, authorization, validation, business logic
Data	PostgreSQL	Vehicle, dealer, customer and service records
Async	Message broker / worker pods	Telemetry/event processing, notifications
Platform	Docker + Kubernetes	Packaging, scaling, rolling deployment, health checks
Observability	Central logs + metrics + tracing	Debugging, alerting, auditability

4. First principles: request lifecycle

1) Client sends HTTPS request with bearer token. 2) Load balancer/API gateway routes to a healthy Kubernetes pod. 3) Django middleware performs common request processing. 4) DRF authentication identifies the caller. 5) Permission classes enforce role/object access. 6) Serializer validates input and converts JSON to Python objects. 7) View/ViewSet invokes service logic. 8) Django ORM reads/writes PostgreSQL. 9) Serializer converts the result to JSON. 10) Response returns through the gateway. Logs, metrics and audit events are emitted without exposing secrets.

5. Suggested domain model

Vehicle(VIN, model, year, owner/customer), Dealer(dealer_code, name), ServiceAppointment(vehicle, dealer, scheduled_at, status), DiagnosticCode(vehicle, code, severity, observed_at), and VehicleEvent(vehicle, event_type, event_time, payload). In a production design, raw telemetry is often separated from transactional tables and may use a streaming/event platform rather than putting high-volume events directly into PostgreSQL.

6. Django + DRF project structure

```
ford_platform/
■■■ manage.py
■■■ requirements.txt
■■■ Dockerfile
■■■ docker-compose.yml
```

```
■■■ k8s/
■   ■■■ deployment.yaml
■   ■■■ service.yaml
■   ■■■ configmap.yaml
■   ■■■ secret.yaml
■■■ config/
■   ■■■ settings.py
■   ■■■ urls.py
■   ■■■ wsgi.py
■■■ vehicle_service/
    ■■■ models.py
    ■■■ serializers.py
    ■■■ views.py
    ■■■ urls.py
    ■■■ permissions.py
    ■■■ tests.py
```

7. Full representative coding skeleton

vehicle_service/models.py

```
from django.db import models

class Dealer(models.Model):
    dealer_code = models.CharField(max_length=20, unique=True)
    name = models.CharField(max_length=120)

class Vehicle(models.Model):
    vin = models.CharField(max_length=17, unique=True, db_index=True)
    model = models.CharField(max_length=80)
    year = models.PositiveSmallIntegerField()
    owner_id = models.CharField(max_length=64, db_index=True)
    dealer = models.ForeignKey(Dealer, on_delete=models.PROTECT)

class ServiceAppointment(models.Model):
    STATUS = [("REQUESTED","Requested"),("CONFIRMED","Confirmed"),
              ("COMPLETED","Completed"),("CANCELLED","Cancelled")]
    vehicle = models.ForeignKey(Vehicle, on_delete=models.CASCADE)
    dealer = models.ForeignKey(Dealer, on_delete=models.PROTECT)
    scheduled_at = models.DateTimeField()
    status = models.CharField(max_length=20, choices=STATUS, default="REQUESTED")
    created_at = models.DateTimeField(auto_now_add=True)

class DiagnosticCode(models.Model):
    vehicle = models.ForeignKey(Vehicle, on_delete=models.CASCADE)
    code = models.CharField(max_length=32)
    severity = models.CharField(max_length=20)
    observed_at = models.DateTimeField()
```

vehicle_service/serializers.py

```
from rest_framework import serializers
from .models import Vehicle, ServiceAppointment, DiagnosticCode

class VehicleSerializer(serializers.ModelSerializer):
    class Meta:
        model = Vehicle
        fields = ["id", "vin", "model", "year", "owner_id", "dealer"]

class ServiceAppointmentSerializer(serializers.ModelSerializer):
    class Meta:
        model = ServiceAppointment
        fields = ["id", "vehicle", "dealer", "scheduled_at", "status", "created_at"]
        read_only_fields = ["created_at"]

    def validate(self, attrs):
        if attrs["vehicle"].dealer_id != attrs["dealer"].id:
            raise serializers.ValidationError(
                "Vehicle is not serviced by the selected dealer."
            )
        return attrs

class DiagnosticCodeSerializer(serializers.ModelSerializer):
    class Meta:
        model = DiagnosticCode
        fields = ["id", "vehicle", "code", "severity", "observed_at"]
```

vehicle_service/permissions.py

```
from rest_framework.permissions import BasePermission
```

```
class IsDealerUser(BasePermission):
    def has_permission(self, request, view):
        return request.user and request.user.is_authenticated

    def has_object_permission(self, request, view, obj):
        # In production, resolve dealer entitlement from the identity/claims.
        return getattr(request.user, "dealer_id", None) == obj.dealer_id
```

vehicle_service/views.py

```
from rest_framework import viewsets
from rest_framework.permissions import IsAuthenticated
from .models import Vehicle, ServiceAppointment, DiagnosticCode
from .serializers import (
    VehicleSerializer, ServiceAppointmentSerializer, DiagnosticCodeSerializer
)

class VehicleViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = Vehicle.objects.select_related("dealer").all()
    serializer_class = VehicleSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        # Scope results to the caller's entitlement.
        qs = super().get_queryset()
        dealer_id = getattr(self.request.user, "dealer_id", None)
        return qs.filter(dealer_id=dealer_id)

class ServiceAppointmentViewSet(viewsets.ModelViewSet):
    queryset = ServiceAppointment.objects.select_related("vehicle", "dealer").all()
    serializer_class = ServiceAppointmentSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        dealer_id = getattr(self.request.user, "dealer_id", None)
        return super().get_queryset().filter(dealer_id=dealer_id)

class DiagnosticCodeViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = DiagnosticCode.objects.select_related("vehicle").all()
    serializer_class = DiagnosticCodeSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        dealer_id = getattr(self.request.user, "dealer_id", None)
        return super().get_queryset().filter(vehicle__dealer_id=dealer_id)
```

vehicle_service/urls.py

```
from rest_framework.routers import DefaultRouter
from .views import VehicleViewSet, ServiceAppointmentViewSet, DiagnosticCodeViewSet

router = DefaultRouter()
router.register("vehicles", VehicleViewSet, basename="vehicle")
router.register("appointments", ServiceAppointmentViewSet, basename="appointment")
router.register("diagnostics", DiagnosticCodeViewSet, basename="diagnostic")

urlpatterns = router.urls
```

config/urls.py

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("api/v1/", include("vehicle_service.urls")),
]
```

config/settings.py

```
import os
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]
DEBUG = os.getenv("DEBUG", "false").lower() == "true"
ALLOWED_HOSTS = os.getenv("ALLOWED_HOSTS", "localhost").split(",")

INSTALLED_APPS = [
    "django.contrib.contenttypes",
    "django.contrib.auth",
    "rest_framework",
```

```
    "vehicle_service",
]

MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
]

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": os.environ["DB_NAME"],
        "USER": os.environ["DB_USER"],
        "PASSWORD": os.environ["DB_PASSWORD"],
        "HOST": os.environ["DB_HOST"],
        "PORT": os.getenv("DB_PORT", "5432"),
    }
}

REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": [
        "rest_framework.authentication.SessionAuthentication",
        # Production example: configure OAuth2/JWT via a vetted identity provider.
    ],
    "DEFAULT_PERMISSION_CLASSES": [
        "rest_framework.permissions.IsAuthenticated",
    ],
    "DEFAULT_THROTTLE_CLASSES": [
        "rest_framework.throttling.UserRateThrottle",
    ],
    "DEFAULT_THROTTLE_RATES": {"user": "1000/day"},
}
SECURE_SSL_REDIRECT = True
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
SECURE_HSTS_SECONDS = 31536000
```

requirements.txt

```
Django>=5.0,<6.0
djangorestframework>=3.15
psycopg[binary]>=3.1
gunicorn>=22
```

Dockerfile

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
RUN useradd --create-home appuser && chown -R appuser:appuser /app
USER appuser

EXPOSE 8000
CMD ["gunicorn", "config.wsgi:application", "--bind", "0.0.0.0:8000", "--workers", "3"]
```

docker-compose.yml

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: ford_platform
      POSTGRES_USER: app
      POSTGRES_PASSWORD: change-me
    volumes:
      - postgres_data:/var/lib/postgresql/data

  web:
    build: .
    environment:
      DJANGO_SECRET_KEY: local-only-secret
      DB_NAME: ford_platform
      DB_USER: app
      DB_PASSWORD: change-me
```

```
      DB_HOST: db
      DB_PORT: "5432"
      ALLOWED_HOSTS: localhost,127.0.0.1
    ports:
      - "8000:8000"
    depends_on:
      - db

volumes:
  postgres_data:
```

k8s/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ford-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: ford-api
  template:
    metadata:
      labels:
        app: ford-api
    spec:
      containers:
      - name: api
        image: registry.example.com/ford-api:1.0.0
        ports:
        - containerPort: 8000
        env:
        - name: DJANGO_SECRET_KEY
          valueFrom:
            secretKeyRef:
              name: ford-api-secret
              key: django-secret
        readinessProbe:
          httpGet:
            path: /health/
            port: 8000
        livenessProbe:
          httpGet:
            path: /health/
            port: 8000
        resources:
          requests:
            cpu: "250m"
            memory: "256Mi"
          limits:
            cpu: "1"
            memory: "512Mi"
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: ford-api-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: ford-api
  minReplicas: 3
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

k8s/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: ford-api
spec:
  selector:
    app: ford-api
```

```
  ports:
  - port: 80
    targetPort: 8000
  type: ClusterIP
```

vehicle_service/tests.py

```
from django.urls import reverse
from rest_framework.test import APITestCase
from django.contrib.auth import get_user_model
from .models import Dealer, Vehicle

class VehicleApiTests(APITestCase):
    def setUp(self):
        self.user = get_user_model().objects.create_user(
            username="dealer_user", password="StrongPassword!"
        )
        self.dealer = Dealer.objects.create(
            dealer_code="D001", name="Example Dealer"
        )
        self.vehicle = Vehicle.objects.create(
            vin="1FTFW1E50PFA00001",
            model="Example Model",
            year=2025,
            owner_id="customer-123",
            dealer=self.dealer,
        )
        self.client.force_authenticate(user=self.user)
        self.user.dealer_id = self.dealer.id

    def test_vehicle_list_requires_authentication(self):
        self.client.force_authenticate(user=None)
        response = self.client.get("/api/v1/vehicles/")
        self.assertEqual(response.status_code, 401)

    def test_vehicle_list_is_scoped(self):
        self.client.force_authenticate(user=self.user)
        # The example demonstrates the API shape; production identity claims
        # should supply dealer entitlement instead of mutating the user object.
        response = self.client.get("/api/v1/vehicles/")
        self.assertIn(response.status_code, [200, 401])
```

8. How to explain REST API security in an interview

Authentication: Use an enterprise identity provider and OAuth2/OIDC. The client obtains an access token; the API validates issuer, audience, signature, expiry and scopes/roles. Avoid saying that the Django application stores passwords for an enterprise integration.

Authorization: Authentication answers “who are you?” Authorization answers “what can you do?” Use DRF permission classes plus object/queryset filtering. A dealer user should only see vehicles and appointments belonging to an authorized dealer.

Input security: DRF serializers validate type, length, allowed values and cross-field rules. Never trust VIN, dealer ID or owner ID supplied by a caller merely because it looks valid.

Transport: HTTPS/TLS end-to-end where possible. Redirect HTTP to HTTPS and use secure cookies/headers where applicable.

Abuse protection: API gateway/WAF, rate limits, request size limits, throttling and anomaly monitoring.

Data protection: Encrypt data at rest, protect database credentials using a secrets manager, use least-privilege DB accounts, avoid logging tokens/PII, and audit sensitive operations.

9. Scaling story

The Django API is designed to be stateless so multiple pods can serve requests. Kubernetes runs several replicas behind a Service. An HPA adds/removes pods based on CPU/memory and, in a mature system, application metrics. PostgreSQL is scaled primarily through indexing, connection pooling, read replicas and query optimization rather than simply adding more API pods. High-volume telemetry is better handled asynchronously through a message broker/stream and worker fleet.

10. Deployment flow

Developer commit → CI runs lint/unit/security checks → build Docker image → vulnerability scan → push immutable image to registry → deploy to Kubernetes → readiness probes verify health → rolling update replaces old pods → metrics/logs monitored → rollback if SLO/error rate degrades. Secrets come from a secret-management mechanism, not from Git.

11. Kubernetes concepts you should be able to explain

Concept	Interview explanation
Pod	Smallest deployable unit; runs one or more containers.
Deployment	Maintains desired replica count and performs rolling updates.
Service	Stable virtual endpoint that routes traffic to matching pods.
Ingress/API Gateway	HTTP routing and edge controls; commonly handles TLS and routing.
ConfigMap	Non-secret configuration.
Secret	Sensitive configuration such as credentials; production should integrate with a proper secret manager.
HPA	Automatically changes replica count based on metrics.
Readiness probe	Determines whether a pod should receive traffic.
Liveness probe	Detects an unhealthy container that needs restart.
Resource requests/limits	Help Kubernetes schedule pods and prevent uncontrolled resource use.

12. Example APIs

```
GET    /api/v1/vehicles/
GET    /api/v1/vehicles/{id}/
GET    /api/v1/vehicles/{id}/diagnostics/
POST   /api/v1/appointments/
GET    /api/v1/appointments/
PATCH  /api/v1/appointments/{id}/
DELETE /api/v1/appointments/{id}/
```

Example request

```
POST /api/v1/appointments/
Authorization: Bearer <access-token>
Content-Type: application/json

{
  "vehicle": 42,
  "dealer": 7,
  "scheduled_at": "2026-09-01T10:30:00Z",
  "status": "REQUESTED"
}
```

13. Interview questions and strong answer points

Q: Why Django REST Framework instead of plain Django?

A: Django provides the web framework and ORM; DRF adds API-oriented serializers, authentication, permissions, throttling, routers, browsable APIs and testing utilities.

Q: What is a serializer?

A: A boundary between external JSON and internal Python/model objects. It validates input and controls exactly which fields can be exposed.

Q: APIView vs ViewSet?

A: APIView gives explicit HTTP-method control. ViewSet groups resource operations and works well with routers for conventional CRUD APIs.

Q: How did you prevent one dealer seeing another dealer's vehicles?

A: Authentication identifies the caller; authorization derives dealer entitlement from trusted identity/context; queryset/object permissions enforce that every read/write is scoped to that dealer.

Q: How did you scale Django?

A: Keep API pods stateless, run multiple replicas, put them behind a Service/load balancer, use HPA, optimize DB access, cache where appropriate and move long-running work to asynchronous workers.

Q: Why Kubernetes?

A: Standardized deployment, service discovery, self-healing, rolling updates, scaling and resource management across many service instances.

Q: What happens if a pod dies?

A: Kubernetes notices the failed pod and the Deployment creates a replacement. Readiness prevents an unready pod from receiving traffic.

Q: How do you avoid DB becoming the bottleneck?

A: Indexes, select_related/prefetch_related, query analysis, pagination, connection pooling, careful transactions, caching and read replicas where justified.

Q: How would you process millions of telemetry events?

A: Do not synchronously write every event through the transactional REST API. Ingest through an event/stream layer, partition by vehicle or event key, process asynchronously, batch writes and keep the API focused on query/command workloads.

Q: How did you handle secrets?

A: No secrets in source control or Docker images. Inject them at deployment time from a secrets-management system and use least privilege.

Q: How did you test?

A: Unit tests for business logic, API tests for status codes/auth/permissions/validation, integration tests for DB behavior, and contract/load/security tests in CI/CD.

Q: How would you debug production latency?

A: Check request metrics and traces, identify slow endpoint/query, inspect DB timings and downstream dependencies, correlate logs using a request/correlation ID, then remediate and verify with metrics.

Q: What is idempotency?

A: Repeating a request should not unintentionally create duplicate effects. For operations such as appointment creation, use an idempotency key and enforce uniqueness at the database/service layer where appropriate.

Q: Why API versioning?

A: It lets consumers migrate safely while the platform evolves. Example: /api/v1/ and /api/v2/.

14. End-to-end implementation sequence

Start with the business use case → define resources and API contracts → create Django project/app → define models and migrations → add serializers → implement ViewSets → add URLs/router → implement authentication/permissions → write API tests → containerize → run locally with Docker Compose → build CI pipeline → deploy to Kubernetes → add probes/HPA → connect managed PostgreSQL → add monitoring/audit/security controls.

15. What to say if asked “Did you personally do all of this?”

Be precise. Separate what you implemented hands-on from what you designed, reviewed or supported. A credible answer is stronger than claiming every platform component. For example: “I owned the Django/DRF API and security layer, worked with the platform team on Docker/Kubernetes deployment, and collaborated with the database and DevOps teams on scaling and production operations.” Adapt this only if it is truthful for your actual experience.

16. Practical commands to remember

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python manage.py makemigrations
python manage.py migrate
python manage.py test
python manage.py runserver

docker build -t ford-api:1.0.0 .
docker compose up --build

kubectl apply -f k8s/
kubectl get pods
kubectl get svc
kubectl describe deployment ford-api
kubectl rollout status deployment/ford-api
kubectl rollout undo deployment/ford-api
```

17. Final mental model

HTTP → Gateway → Django/DRF → AuthN → AuthZ → Serializer → View/Service → ORM → PostgreSQL → JSON response
Then: **Docker packages it; Kubernetes runs/scales it; CI/CD deploys it; monitoring tells you whether it is healthy.**